ICE Task 3: Pipelines and Text Data

Parts 1 and 2

Model selection, dataset justification and analysis plan

Amazon Fine Food Reviews: finding the themes behind the stars

Prepared by: Sifiso Mahlangu **Module:** PDAN8411, Programming for Data Analytics **Date:** 02 October 2026

The client sells food on Amazon and has noticed that customer feedback is getting more varied. It wants to know two things: what customers are unhappy or happy about, and how they feel overall. Those are two different problems, so I use two different models. This document explains which ones, why, and how the analysis will run. The results sit in the final report and the notebook.

Part 1: Model selection and dataset justification

1.1 Model selection

The first model finds themes without being told what to look for (unsupervised). The second learns from star ratings to predict whether a review is positive or negative (supervised).

	Themes (concern and praise)	Sentiment
Chosen model	Latent Dirichlet Allocation (LDA), fitted separately on negative and on positive reviews, backed by a keyword count of business topics (taste, packaging, delivery, price, freshness)	Logistic Regression on TF-IDF word and two-word-phrase features
Tested or considered	NMF and K-means (considered); plain keyword extraction (used as a cross-check through distinctive-word counts)	Naive Bayes and Linear SVM (both trained and compared); LSTM and Transformer models (considered, not run)
What it produces	A short list of topics per group. Each topic has its top words and its share of reviews	A positive or negative label and a probability for every review, plus the words that drove the decision

Themes: Latent Dirichlet Allocation (LDA)

Why it suits text. LDA treats every review as a mix of topics, and every topic as a mix of words. That matches how people write reviews. One review can talk about taste and shipping together, and LDA lets it belong partly to both. It also needs no labels, which matters because nobody has tagged these reviews with themes.

Strengths	Limitations	Expected output
- Learns topics straight from the words, no labelling needed - A review can carry more than one theme - Topics read as plain word lists a manager can understand - Online learning copes with hundreds of thousands of reviews	- I must choose the number of topics (I pick it by coherence score, see 2.3) - A person must name each topic from its words - Short, informal reviews give noisy topics - It shows what reviews are about, not whether it is good or bad, so I fit it separately on negative and positive reviews - Product types (coffee, dog food) can crowd out complaint types, so I add the keyword count of business topics	- For each group: the chosen number of topics, each with its 10 top words and its share of reviews - A coherence score for every topic count tried - Named themes written up in the report

Why not the alternatives. K-means forces every review into one cluster, and its cluster centres are harder to read. NMF gives similar topics and is a good cross-check, but LDA is the standard method for this job, and it is what the brief names. Plain keyword extraction lists frequent words, not themes. BERTopic (embedding-based) usually gives cleaner topics but needs a GPU and much more compute than I have here.

Sentiment: Logistic Regression on TF-IDF features

Why it suits text. Once reviews are turned into TF-IDF numbers, each one is a very long row where almost every value is zero. Linear models handle that shape well. They train quickly, they are hard to overfit on sparse data, and every word gets a weight I can read, so I can tell the client which words push a review towards negative. Logistic Regression also gives a probability, so the client can decide how cautious to be when flagging a complaint.

Strengths	Limitations	Expected output
- Fast, even on 300,000+ reviews - Every word weight can be read and explained - Gives a probability, not just a label - A strong, standard baseline for text - Accepts class weights for imbalanced data	- Counts words, does not understand meaning: sarcasm and mixed reviews ("great taste, too expensive") can be misread - Two-word phrases give only a little context - With few negatives it leans towards the majority class unless weighted - Star ratings are a noisy stand-in for true sentiment	- A label and probability for each review - Accuracy, precision, recall, F1 and a confusion matrix - The strongest praise and complaint words

Why not the alternatives. Naive Bayes is the fastest but assumes words are independent, and it is usually weaker, so I train it as a comparison. Linear SVM is a close cousin of Logistic Regression and I train it too, but it gives no probabilities out of the box. LSTM and Transformer models read word order and context, so they should handle sarcasm and mixed reviews better. Training one on 300,000+ reviews realistically need a GPU, and I have two CPU cores. I treat them as the next step if the client needs more accuracy than a linear model gives. I have not run them, so I make no claim about how much better they would be.

1.2 Dataset justification

I use the Amazon Fine Food Reviews dataset (released by Stanford's SNAP group on Kaggle). The file I received has 568,454 reviews. These figures come from running the audit in the notebook.

Criterion	What I found	What I do about it
Relevance	Real Amazon food product reviews. Each row has the review text, a summary line, a 1-to-5-star rating, product and reviewer IDs, helpfulness votes and a date. This is the scenario in the brief almost word for word.	Use the stars as a ready-made sentiment label, so nothing must be labelled by hand.
Data quality: missing values	Review text: 0 missing. Summary: 27 missing. Profile name: 26 missing.	Keep reviews with a missing summary (empty summary). Ignore profile name, which is not used.
Data quality: duplicates	174,562 rows (30.7%) are the same reviewer posting the same text at the same time. These are one review copied across product variants. 2 more rows have more helpful votes than total votes, which is impossible.	Remove the copies and the impossible rows. Left in, copies would inflate counts and put identical text in both the training and test sets.
Data quality: noise	Text contains HTML tags (br), web links, typos, capital-letter shouting and star ratings that disagree with the text.	Normalize the text (see 2.1). Treat star-based labels as a good but imperfect proxy.
Size and diversity	568,454 raw reviews, 74,258 products, 256,059 reviewers, from 8 October 1999 to 26 October 2012. After cleaning, 393,888 reviews remain. Products span coffee, tea, snacks, baby food and pet food.	Large enough to train, hold back a test set and fit topic models. Watch the skew: 64% of reviews are five stars, only 15.7% of reviews with a clear sentiment are negative, and about 64% of the clean reviews come from 2011 and 2012.
Ethics: bias and fairness	Only people who chose to review are included, and they tend to be very happy or very unhappy. There is no way to check who bought the product, or to spot paid or fake reviews. The data has no demographics, so I cannot test fairness across customer groups. A model trained on one style of English can misread slang or non-native phrasing.	Say plainly in the report that results describe reviewers, not all customers. Use scores in aggregate and have a person check before acting on any single customer. Record the dataset age (2012) as a limit.
Ethics: privacy and terms	Reviewer IDs and profile names are personal data in a public file.	Do not use profile names, do not try to identify anyone, report only aggregates. Confirm Amazon's terms before using this data beyond coursework.

What the dataset does not have. There are no product names (only IDs), no verified-purchase flag and no delivery or courier data. So, I can say that reviews mention packaging problems, but I cannot say which brand or which courier causes them.

Part 2: Planning the analysis

The analysis runs as one pipeline. Each step feeds the next, and the test reviews stay out of every decision about settings.



Figure 1: The analysis pipeline, from raw reviews to the client report.

2.1 Exploratory data analysis plan

Step	Action	Why
Missing values	Keep reviews with no summary (use an empty summary). Ignore the profile name. Review text is never missing.	Losing a good review over a missing title wastes data.
Impossible values	Drop rows where helpful votes exceed total votes.	They signal bad records.
Duplicates	Drop repeated reviews with the same reviewer, time and text. Keep the first.	Prevents inflated counts and train/test leakage.
Sentiment label	4 to 5 stars positive, 1 to 2 stars negative. 3 stars are neutral: left out of training, scored afterwards.	Three-star reviews are mixed and blur the line between classes.
Lowercase	Convert all text to lowercase.	"Great" and "great" are the same word.
Remove HTML and links	Strip tags such as br and any web addresses.	Pure noise.
Contractions	Turn "didn't" into "did not" and "can't" into "cannot".	Stops a stray "t" being left behind and keeps the negation.
Punctuation and digits	Remove them.	Gives clean words. Trade-off: exclamation marks carry some feeling, which I give up for simplicity.
Stop words	Remove common English filler words, but keep not, no, never, nothing, without and similar negations.	"Not good" must stay different from "good".
Empty reviews	Drop reviews with nothing left after cleaning.	Nothing to model.

Planned EDA outputs: star-rating distribution, reviews and negative share by year, review length by sentiment, most common words, word clouds, words that separate complaints from praise, and a risk chart for business topics.

2.2 Feature selection plan

A model cannot read words, so each review becomes numbers. I compare four options on a validation slice cut from the training data, and I tune the regularization strength for each option so none is handicapped by its scaling.

Option	What it does	Plus	Minus
Bag of words (BoW)	Counts how often each word appears	Simple, fast	Long reviews and common words dominate
TF-IDF	Counts words, then down-weights words that appear in many reviews	Highlights words that characterize a review	Still ignores word order
N-grams (1 to 2 words)	Adds two-word phrases such as "not good" and "too sweet"	Captures a little context and negation	Many more features, so more memory

Candidates compared: (1) BoW single words, 20,000 features; (2) TF-IDF single words, 20,000; (3) TF-IDF with 1-to-2-word phrases, 20,000; (4) the same with 100,000 features. Common settings: words must appear in at least 5 reviews (min_df=5) and term counts are log-scaled (sublinear TF). The winner informs the improved model.

2.3 Model training plan

Item	Plan
Train/test split	80% train, 20% test. Stratified so both keep the same positive/negative mix. Random seed 42 so anyone can reproduce it.
Neutral reviews	3-star reviews are excluded from training and testing. The final model scores them afterwards to see where they land.
Validation	A slice of about 120,000 training reviews (80/20 inside it) is used to compare features and tune settings. The test set is never used to choose settings.
Baseline (settings fixed in advance)	TF-IDF single words, max_features 20,000, min_df 5. Logistic Regression C=1.0. Naive Bayes alpha=1.0. Plus, a majority-class guess to show what accuracy alone hides.
Tuning grid	Logistic Regression: C in 0.5, 1, 5, 20, each with and without balanced class weights. Linear SVM: C in 0.1, 0.3, 1, each with and without balanced weights. Naive Bayes: alpha in 0.05, 0.1, 0.3, 1.0. Best setting per model is the one with the highest macro-F1 on the validation slice.
Improved features	TF-IDF, 1-to-2-word phrases, max_features 200,000 (larger than the 100,000 compared, because the full training set holds more phrases than the validation slice).
Epochs and learning rate	Not applicable. Logistic Regression is fitted by an optimizer (lbfgs, up to 1,000 iterations) and not trained in epochs. These settings would apply if I moved to an LSTM or Transformer.
Topic model (LDA)	Fitted separately on negative and on positive reviews. 40,000 reviews sampled per group, vocabulary of 6,000 words, online learning, 8 passes, batch size 2,048. Topic counts tried: 4, 6, 8, 10. The count with the best NPMI coherence is chosen. Generic filler words are removed for this step only.

2.4 Evaluation plan

Metric	What it tells me	Why it matters here
Accuracy	Share of reviews labelled correctly	Misleading on its own: a model that always says "positive" scores high because most reviews are positive.
Precision	When the model says "negative", how often it is right	Wrongly flagging happy customers wastes the client's time.
Recall	Of the truly negative reviews, how many the model finds	Missed complaints are missed chances to fix problems. This is the weak spot to watch.
F1-score	Balance of precision and recall	One number per class. The negative-class F1 is my headline metric, with macro-F1 (average of both classes) as the overall check.
Confusion matrix	The four outcomes laid out in a grid	Shows exactly which mistakes the model makes.
ROC AUC	How well the model ranks negatives above positives at any threshold	Shows whether a better threshold could help.

Decision rule, set before running: the improved model counts as better only if it beats the baseline on both macro-F1 and negative-class F1 on the held-out test set. I also run three checks: a majority-class baseline, a time-based split (train on older reviews, test on the newest) and a review of the actual mistakes.

2.5 Reporting plan

Report section	Content	Visuals
Introduction	The business question and what the client gets	None
Dataset overview	What the data is and its limits	Score distribution
Data cleaning and EDA	What I removed and fixed. What the reviews look like	Trend by year, review length, word frequency, word clouds, distinctive words
Feature engineering	How text becomes numbers, in plain words, and why this choice	Feature comparison table
Model training	Baseline and improved model settings	Settings table
Model evaluation	Results, strengths, weaknesses, business meaning, improvement impact	Confusion matrices, model comparison, key words
Key findings	Themes of concern and praise, satisfaction trend	Aspect risk chart, topic charts
Recommendations	Specific actions in priority order, each tied to evidence	Action table

Risks and how I handle them

Risk	Handling
Duplicates leak between train and test and inflate scores	Remove duplicates before splitting.
Few negative reviews make the model favour "positive"	Test balanced class weights in the tuning grid, report scores for each class and compare against a majority-class guess.
Sarcasm and mixed reviews	Review real errors, report them honestly, and flag a Transformer model as the next step.
Old data (to 2012) may not reflect today	Run a time-based split and state the age of the data as a limit.
Topics driven by product type rather than complaint type	Fit topics separately on negative and positive reviews and add the keyword business-topic count.